



AKADEMIA GÓRNICZO-HUTNICZA IM. STANISŁAWA STASZICA W KRAKOWIE

WYDZIAŁ INŻYNIERII METALI I INFORMATYKI PRZEMYSŁOWEJ

KATEDRA INFORMATYKI STOSOWANEJ I MODELOWANIA

Projekt dyplomowy

Wykorzystanie technik DevOps w implementacji automatycznego procesu CI/CD do uczenia i wersjonowania modeli sztucznej inteligencji: analiza utrzymania systemów wykorzystujących sztuczną inteligencję w porównaniu do metod manualnych

Using DevOps techniques to implement an automated CI/CD process for training and versioning AI models: an analysis of the maintenance of AI-based systems compared to manual methods

Autor: Rafał Zabłotni
Kierunek studiów: Inżynieria Obliczeniowa
Opiekun pracy: mgr inż. Piotr Hajder

Kraków, 2026

Spis treści

1	Wstęp	3
1.1	Geneza	3
1.2	Założenia	4
2	Wprowadzenie Teoretyczne	5
2.1	DevOps	5
2.2	Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)	5
2.3	MLOps	6
2.4	Kontrola wersji — Git i GitHub	7
2.5	GitHub Actions	7
2.6	Konteneryzacja — Docker	8
2.7	Kubernetes i k3s	8
2.8	Infrastruktura jako kod — Terraform i Helm	8
2.9	Wersjonowanie danych — DVC	9
2.10	Śledzenie eksperymentów — MLflow	9
2.11	Monitoring — Prometheus i Grafana	9
3	Architektura Systemu	10
3.1	Cele i ograniczenia projektowe	10
3.2	Struktura repozytoriów	10
3.3	Warstwy architektury	11
3.3.1	Warstwa kontroli wersji	13
3.3.2	Warstwa automatyzacji	13
3.3.3	Warstwa infrastruktury chmurowej	13
3.3.4	Warstwa aplikacji i monitoringu	13
3.4	Kluczowe decyzje projektowe	13
4	Model Sztucznej Inteligencji	15
4.1	Charakterystyka problemu segmentacji semantycznej	15
4.2	Architektura modelu	15
4.3	Dane treningowe i przygotowanie zbioru danych	16
4.4	Proces uczenia i metryki jakości	17
4.5	Model jako artefakt w procesie CI/CD	17
5	Pipeline CI/CD	18
5.1	Hook pre-push	18
5.2	Walidacja danych	18
5.3	Pipeline treningowy	20
5.4	Współbieżność i kolejkowanie	22
5.5	Bramka jakości	22

5.6	Automatyczne wdrożenie	23
6	Wdrożenie Systemu	24
6.1	Infrastruktura Terraform	24
6.2	Konfiguracja EC2 i k3s	24
6.3	Konfiguracja Helm	25
6.4	Stos monitorowania	25
6.5	Workflow użytkownika	26
6.5.1	Scenariusz 1: Dodanie nowych danych treningowych	26
6.5.2	Scenariusz 2: Predykcja lokalna	26
6.5.3	Scenariusz 3: Ręczne uruchomienie treningu	26
7	Analiza Wyników i Działania Systemu WIP	27
7.1	Metodologia porównania	27
7.2	Czasy wykonania scenariuszy	27
7.3	Metryki modelu	27
7.4	Metryki infrastruktury	28
7.4.1	Zużycie zasobów podczas treningu	28
7.4.2	Zużycie zasobów podczas serwowania	28
7.4.3	Koszty infrastruktury AWS	28
7.5	Ocena jakościowa	28
7.5.1	Łatwość wdrożenia	28
7.5.2	Stabilność systemu	28
7.5.3	Skalowalność	28
8	Podsumowanie i wnioski WIP	29
8.1	Realizacja celów pracy	29
8.2	Wnioski	29
8.3	Ograniczenia	29
8.4	Kierunki dalszego rozwoju	30
	Literatura	31

1 Wstęp

1.1 Geneza

Pomysł na realizację niniejszego projektu zrodził się podczas wykonywania codziennych obowiązków zawodowych. W trakcie pracy nad jednym z projektów zaobserwowano interesującą tendencję związaną z procesem tworzenia testów modułowych. Znaczna część kodu testowego okazywała się redundantna, a wiele mechanizmów było wielokrotnie powielanych poprzez kopiowanie oraz wprowadzanie drobnych modyfikacji do istniejących fragmentów kodu.

Naturalnym rozwiązaniem tego problemu mogłoby wydawać się wydzielanie powtarzalnych elementów do funkcji pomocniczych lub bibliotek testowych oraz utrzymywanie ich w uporządkowanej i współdzielonej formie. Podejście to jednak wiąże się z istotnymi trudnościami, zarówno natury organizacyjnej, jak i technicznej. W praktyce często prowadzi ono do nadmiernego rozrostu bibliotek pomocniczych, niejednoznacznych interfejsów oraz problemów z ich długoterminowym utrzymaniem.

W szczególności pojawiają się następujące problemy:

1. Jak określić moment, w którym dana funkcjonalność jest wystarczająco generyczna, aby mogła zostać wydzielona do wspólnej biblioteki?
2. Jak zminimalizować ryzyko tworzenia funkcji lokalnie w plikach testowych, które z czasem zaczynają być wykorzystywane przez inne testy, prowadząc do niejawnych zależności pomiędzy teoretycznie niezależnymi modułami?
3. Jak zapewnić spójny sposób tworzenia oraz utrzymania takich funkcji w projektach rozwijanych przez dziesiątki, a niekiedy setki programistów, o różnym poziomie doświadczenia i odmiennych nawykach pracy?

Jednocześnie, jak powszechnie wiadomo, modele sztucznej inteligencji uczą się na podstawie zbiorów danych treningowych, a następnie wykorzystują wyuczone wzorce do rozwiązywania nowych problemów. Obserwacja ta stała się punktem wyjścia do rozważenia możliwości zastosowania modeli AI w obszarze automatycznego generowania wspólnych, powtarzalnych fragmentów kodu testowego na podstawie już istniejących przykładów.

Jednakże, aby taki system mógł być skutecznie wykorzystany w praktyce, konieczne jest zapewnienie odpowiedniego procesu ciągłej integracji i dostarczania (CI/CD), który umożliwi automatyczne trenowanie, ocenę jakości oraz wdrażanie modeli AI w sposób powtarzalny i kontrolowany, lub oddelegowanie pracownika do utrzymania takiego procesu w sposób manualny.

W ten sposób pojawiają dwa kluczowe pytania, które stanowią oś projektu:

1. Czy możliwe i opłacalne jest stworzenie modelu SI który będzie obsługiwał automatyczne generowanie fragmentów kodu testowego?
2. Czy bardziej opłacalne jest oddelegowanie jednego pracownika do ręcznego utrzymywania i rozwijania tego oprogramowania, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps oraz pipeline'ów CI/CD?

1.2 Założenia

Ze względu na złożoność oryginalnego problemu generowania kodu testowego oraz ograniczony czas realizacji projektu, zdecydowano się na zastosowanie problemu zastępczego, który pozwala w pełni zweryfikować hipotezę bez konieczności budowania zaawansowanego modelu językowego. Jako taki substytut wybrano segmentację semantyczną obrazów wodomierzy z wykorzystaniem architektury U-Net. Wybór ten jest celowy: model segmentacji obrazów posiada wyraźnie zdefiniowane metryki jakości (współczynnik Dice, IoU), umiarkowane wymagania obliczeniowe umożliwiające trening w ramach dostępnego budżetu, a dostępność wstępnie wytrenowanego modelu bazowego pozwala na obiektywne porównanie kolejnych wersji. Dla uproszczenia przyjęto, że model SI dostarczany jest przez zewnętrzną firmę, natomiast zakres pracy obejmuje jego dalsze uczenie, ocenę jakości oraz wdrażanie. Takie założenie odzwierciedla realny scenariusz w środowisku produkcyjnym, gdzie zespół DevOps otrzymuje gotowy model i odpowiada za zarządzanie jego cyklem życia.

W tak zdefiniowanym kontekście drugie pytanie sformułowane w poprzedniej sekcji pozostaje w pełni aktualne: czy bardziej opłacalne jest oddelegowanie pracownika do ręcznego utrzymywania i rozwijania modelu, czy też lepszym rozwiązaniem jest automatyzacja tego procesu z wykorzystaniem narzędzi DevOps i pipeline'ów CI/CD? W podejściu manualnym proces ponownego uczenia i wdrażania modelu wymaga ręcznego przygotowania danych, uruchamiania treningu, walidacji wyników oraz publikacji nowej wersji. Takie podejście, choć proste koncepcyjnie, wiąże się z ryzykiem błędów ludzkich, trudnościami w zachowaniu powtarzalności eksperymentów oraz ograniczoną skalowalnością. Automatyzacja natomiast umożliwia jednoznaczne powiązanie wersji modelu z wersją danych treningowych, wprowadzenie obiektywnej bramki jakości i eliminację ręcznych kroków, które mogą prowadzić do niespójności.

W ramach niniejszej pracy opracowano kompletny, zautomatyzowany system CI/CD dla modeli sztucznej inteligencji. System obejmuje wersjonowanie danych treningowych (DVC), śledzenie eksperymentów i rejestr modeli (MLflow), automatyczny pipeline treningowy uruchamiany przez GitHub Actions, bramkę jakości porównującą nowy model z aktualną wersją produkcyjną, a także wdrożenie modelu w środowisku chmurowym (k3s na EC2) z możliwością użycia aplikacji oraz monitoringiem (Prometheus, Grafana). Całość zaprojektowano tak, aby dodanie nowych danych treningowych przez użytkownika wymagało jedynie wykonania `git push`, a dalszy proces odbywał się bez jego udziału. Tak zdefiniowany zakres pozwala na bezpośrednie porównanie kosztów i nakładu pracy podejścia manualnego i automatycznego, co stanowi główny cel analityczny pracy.

2 Wprowadzenie Teoretyczne

Realizacja projektu opisanego w niniejszej pracy opiera się na zestawie technologii i koncepcji z obszaru inżynierii oprogramowania, uczenia maszynowego i infrastruktury chmurowej. Niniejszy rozdział przedstawia teoretyczne podstawy każdego z tych elementów, stanowiąc punkt odniesienia dla decyzji projektowych opisanych w kolejnych rozdziałach. Omówiono kolejno: filozofię DevOps i mechanizmy CI/CD, podejście MLOps jako ich rozszerzenie na świat modeli ML, a następnie konkretne narzędzia zastosowane w projekcie: Git i GitHub, GitHub Actions, Docker, Kubernetes (k3s), Terraform z Helm, MLflow, DVC oraz Prometheus z Grafaną.

2.1 DevOps

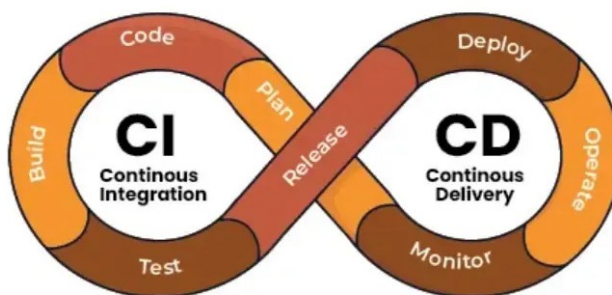
DevOps to zbiór praktyk, kultury organizacyjnej i narzędzi, których celem jest skrócenie cyklu życia oprogramowania przy jednoczesnym zapewnieniu wysokiej jakości dostarczanych produktów. Termin pochodzi od połączenia słów *Development* i *Operations*, co odzwierciedla dążenie do przełamania tradycyjnej bariery między zespołami deweloperskimi a operacyjnymi.

Kluczowym elementem filozofii DevOps jest pętla ciągłego doskonalenia, obejmująca planowanie, kodowanie, budowanie, testowanie, wdrażanie, uruchamianie oraz monitorowanie. Każdy obrót pętli dostarcza informacje zwrotne, które pozwalają szybciej reagować na błędy i wymagania użytkowników. W kontekście niniejszej pracy DevOps stanowi ramy koncepcyjne, w których osadzony jest cały projekt - automatyzacja procesu trenowania i wdrażania modelu SI jest bezpośrednią aplikacją tych zasad [13].

2.2 Ciągła Integracja i Ciągłe Dostarczanie (CI/CD)

Ciągła Integracja (ang. *Continuous Integration*, CI) polega na automatycznym scalaniu i weryfikacji zmian kodu wielokrotnie w ciągu dnia. Każda zmiana uruchamia zdefiniowany zestaw testów, co pozwala wykryć błędy integracyjne natychmiast po ich wprowadzeniu, zamiast odkrywać je dopiero na etapie wydania.

Ciągłe Dostarczanie (ang. *Continuous Delivery*) rozszerza CI o automatyczne przygotowanie artefaktów gotowych do wdrożenia. Ciągłe Wdrażanie (ang. *Continuous Deployment*) idzie krok dalej - każda zmiana spełniająca kryteria jakości jest wdrażana do produkcji.



Rys. 1 Przykład flow CI/CD [15].

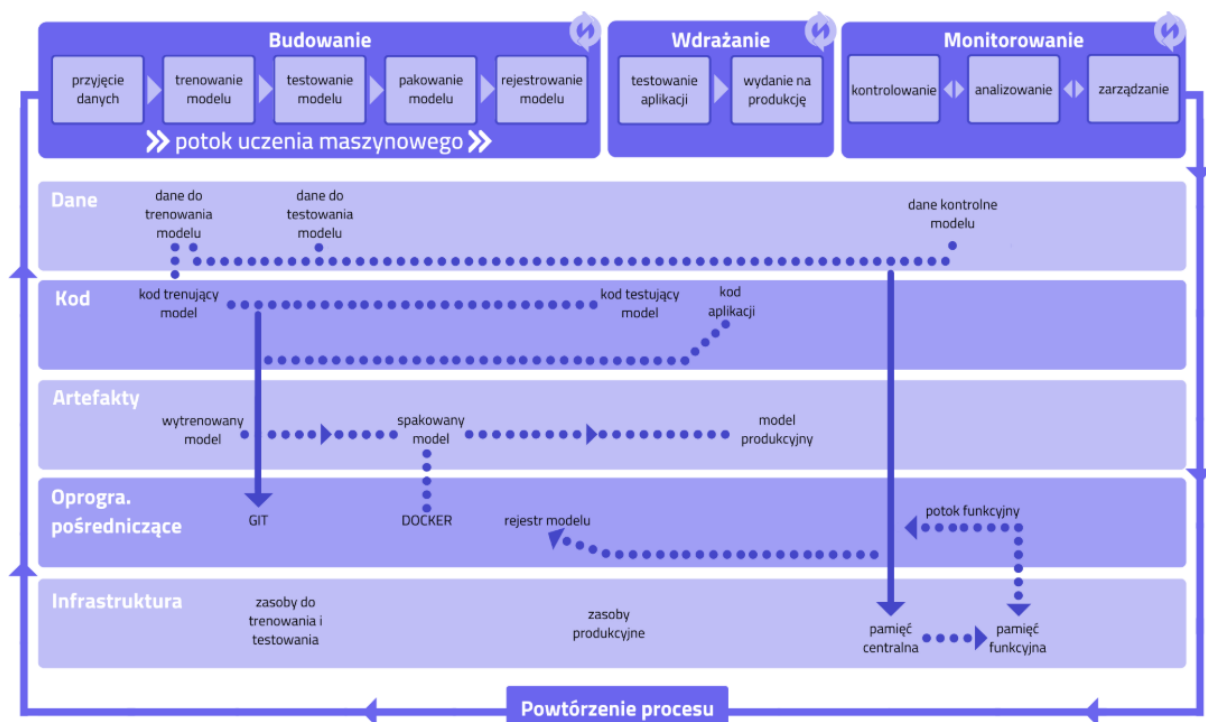
Typowy pipeline CI/CD składa się z następujących etapów:

1. weryfikacja kodu (linting, testy jednostkowe),
2. budowanie artefaktów (np. obrazu Docker),
3. testy integracyjne i akceptacyjne,
4. wdrożenie na środowisko docelowe,
5. weryfikacja po wdrożeniu (smoke tests).

2.3 MLOps

MLOps (ang. *Machine Learning Operations*) to dyscyplina stosująca zasady DevOps do specyfikacji cyklu życia modeli uczenia maszynowego. W odróżnieniu od klasycznego oprogramowania, gdzie artefaktem jest kod, w ML artefaktem jest model — wynik trenowania na konkretnym zbiorze danych. To wprowadza kilka dodatkowych wyzwań:

- dane treningowe zmieniają się niezależnie od kodu,
- wyniki trenowania są niedeterministyczne,
- jakość modelu musi być mierzona obiektywnie przed każdym wdrożeniem,
- modele degradują się w czasie (tzw. *data drift*).



Rys. 2 Przykład flow CI/CD [5].

MLflow spina cały cykl życia modelu ML — od rejestrowania eksperymentów, parametrów i metryk podczas treningu, przez wersjonowanie oraz rejestrację modeli w Model Registry, aż po zarządzanie wdrożeniem i monitorowanie kolejnych wersji w środowisku produkcyjnym.

2.4 Kontrola wersji — Git i GitHub

Git to rozproszony system kontroli wersji, będący de facto standardem w nowoczesnym tworzeniu oprogramowania [3]. Przechowuje pełną historię zmian w postaci grafu migawek (ang. *snapshots*), umożliwiając równoległą pracę na gałęziach (ang. *branches*), scalanie zmian oraz cofanie się do dowolnego punktu w historii projektu.

Kluczowe koncepcje Gita:

- **Commit** — migawka stanu repozytorium w danym momencie wraz z metadanymi (autor, czas, wiadomość),
- **Branch** — niezależna linia rozwoju; umożliwia pracę nad funkcjonalnością bez wpływu na gałąź główną,
- **Merge / Pull Request** — scalenie zmian z jednej gałęzi do drugiej, zazwyczaj poprzedzone przeglądem kodu,
- **Hook** — skrypt uruchamiany automatycznie w odpowiedzi na zdarzenie Gita (np. *pre-push*).

GitHub to platforma hostingowa dla repozytoriów Git, rozbudowana o funkcje współpracy zespołowej: pull requesty, przeglądy kodu, zarządzanie zadaniami oraz zintegrowane CI/CD w postaci GitHub Actions. W kontekście niniejszego projektu GitHub pełni rolę centralnego punktu integracji — każda zmiana przechodzi przez repozytorium i może automatycznie uruchamiać kolejne etapy pipeline'u.

2.5 GitHub Actions

GitHub Actions to platforma automatyzacji wbudowana w GitHub, pozwalająca definiować przepływy pracy (ang. *workflows*) jako pliki YAML przechowywane bezpośrednio w repozytorium [6]. Workflow uruchamiane są w odpowiedzi na zdarzenia: push, pull request, harmonogram (cron) lub wywołanie ręczne.

Podstawowe pojęcia:

- **Workflow** — zestaw automatycznych zadań zdefiniowany w pliku `.github/workflows/*.yaml`,
- **Job** — jednostka wykonania złożona z kroków (*steps*); domyślnie uruchamiana równoległe z innymi jobami,
- **Step** — pojedyncza operacja w ramach joba: wywołanie skryptu lub gotowej akcji,
- **Runner** — maszyna wykonująca joba; może być zarządzana przez GitHub (*hosted*) lub przez użytkownika (*self-hosted*),
- **Artefakt** — plik lub katalog przekazywany między jobami albo pobierany po zakończeniu workflow.

Kluczową cechą GitHub Actions jest możliwość użycia **self-hosted runners** — maszyn zarządzanych przez użytkownika, co w niniejszym projekcie pozwala uruchamiać trening bezpośrednio na instancji EC2 wyposażonej w odpowiedni sprzęt obliczeniowy, bez potrzeby przekazywania danych treningowych przez sieć do zewnętrznego runnera.

2.6 Konteneryzacja — Docker

Docker to platforma do tworzenia, dystrybucji i uruchamiania aplikacji w kontenerach [14]. Kontener to izolowane środowisko uruchomieniowe zawierające aplikację wraz ze wszystkimi jej zależnościami: bibliotekami, interpreterem, zmiennymi środowiskowymi. Dzięki temu aplikacja działa identycznie na maszynie dewelopera, serwerze testowym i produkcji, eliminując problem „u mnie działa”.

Kluczowe pojęcia:

- **Obraz (image)** — niezmienny szablon opisujący środowisko, definiowany plikiem `Dockerfile`,
- **Kontener** — działająca instancja obrazu; izolowana od systemu hosta i innych kontenerów,
- **Rejestr (registry)** — repozytorium obrazów (np. Docker Hub, Amazon ECR).

W projekcie Docker służy do pakowania API modelu oraz do zapewnienia powtarzalności środowiska treningowego. Obraz z modelem jest budowany automatycznie w pipeline po zatwierdzeniu nowej wersji modelu.

2.7 Kubernetes i k3s

Kubernetes (K8s) to system do orkiestracji kontenerów — automatyzuje ich wdrażanie, skalowanie i zarządzanie [2]. Zamiast uruchamiać kontenery ręcznie na konkretnych maszynach, Kubernetes przyjmuje deklaracyjny opis pożądanego stanu (ile replik, jakie zasoby, jak reagować na awarie) i utrzymuje ten stan niezależnie od zdarzeń infrastrukturalnych.

k3s to uproszczona dystrybucja Kubernetes stworzona przez Rancher Labs. Zmniejsza wymagania sprzętowe (minimalna instalacja działa na instancji z 2 GB RAM) i upraszcza instalację do pojedynczego polecenia. Jest przeznaczona dla środowisk brzegowych, IoT oraz małych klastrów. W niniejszym projekcie k3s zainstalowany na pojedynczej instancji EC2 zastępuje zarządzany klaster EKS, co pozwala ograniczyć koszty do minimum.

2.8 Infrastruktura jako kod — Terraform i Helm

Infrastruktura jako kod (ang. *Infrastructure as Code*, IaC) to praktyka definiowania i zarządzania zasobami infrastrukturalnymi za pomocą plików konfiguracyjnych przechowywanych w systemie kontroli wersji, zamiast ręcznego konfigurowania przez interfejs graficzny lub CLI [1]. Dzięki temu środowisko jest powtarzalne, audytowalne i odtwarzalne dowolnym momencie.

Terraform to narzędzie IaC firmy HashiCorp umożliwiające tworzenie, modyfikowanie i niszczenie zasobów chmurowych w deklaracyjnym języku HCL (ang. *HashiCorp Configuration Language*). Terraform utrzymuje plik stanu (`terraform.tfstate`) opisujący aktualną konfigurację infrastruktury, co pozwala wykrywać i stosować wyłącznie te zmiany, które są niezbędne. W projekcie Terraform zarządza zasobami AWS: siecią VPC, instancją EC2, zasobnikami S3 i rejestrem ECR.

Helm to menedżer pakietów dla Kubernetes, analogiczny do apt czy pip w świecie aplikacji [7]. Pakiet Helm (ang. *chart*) zawiera szablony zasobów Kubernetes parametryzowane warto-

ściami z pliku `values.yaml`. Umożliwia to instalację, aktualizację i usuwanie złożonych aplikacji wieloskładnikowych jednym poleceniem, z pełną historią wdrożeń i możliwością cofnięcia (`rollback`). W projekcie Helm zarządza wdrożeniem API modelu oraz stosu monitorującego na klastrze k3s.

2.9 Wersjonowanie danych — DVC

DVC (ang. *Data Version Control*) to narzędzie umożliwiające wersjonowanie dużych plików binarnych (obrazów, wag modeli, zbiorów danych) w sposób spójny z Git. Zamiast przechowywać dane w repozytorium Git (co jest niepraktyczne przy dużych plikach), DVC przechowuje jedynie lekkie pliki metadanych (`.dvc`), a rzeczywiste dane trafiają do zewnętrznego magazynu — w tym projekcie do Amazon S3 [9].

Zasada działania jest analogiczna do Git: każda wersja danych ma swój hash, można cofać się do poprzednich wersji poleceniem `dvc checkout`, a polecenie `dvc pull` pobiera dane odpowiadające bieżącemu stanowi repozytorium. Dzięki temu możliwe jest odtworzenie dokładnego eksperymentu: tej samej wersji kodu, danych i konfiguracji.

2.10 Śledzenie eksperymentów — MLflow

MLflow to platforma open-source do zarządzania cyklem życia modeli ML. Składa się z czterech głównych komponentów:

- **Tracking** — rejestrowanie parametrów, metryk i artefaktów każdego uruchomienia eksperymentu,
- **Projects** — standaryzacja i reprodukowalność kodu ML,
- **Models** — zunifikowany format pakowania modeli,
- **Model Registry** — centralny rejestr wersji modeli z etapami cyklu życia (Staging, Production, Archived).

Model Registry jest kluczowym elementem w kontekście bramki jakości. Po każdym treningu nowa wersja modelu jest rejestrowana w MLflow. Bramka jakości pobiera metryki modelu aktualnie w etapie Production i porównuje z nowym wynikiem. Tylko model z lepszymi metrykami zostaje awansowany do Production [16].

2.11 Monitoring — Prometheus i Grafana

Prometheus to system monitorowania i alertowania oparty na modelu *pull* [17]. Regularnie pobiera (ang. *scrape*) metryki z endpointów HTTP udostępnianych przez monitorowane usługi i przechowuje je w wbudowanej bazie szeregów czasowych. Metryki opisywane są etykietami (ang. *labels*), co pozwala na elastyczne filtrowanie i agregację.

Grafana to platforma wizualizacji danych, która integruje się z wieloma źródłami, w tym z Prometheus. Pozwala tworzyć interaktywne dashboardy z wykresami, wskaźnikami i alertami. W projekcie Grafana prezentuje metryki działającego API modelu: liczbę predykcji, czasy odpowiedzi, zużycie zasobów i liczbę błędów. Monitoring działa jako opcjonalny stos uruchamiany na tym samym klastrze k3s co API modelu [12].

3 Architektura Systemu

Niniejszy rozdział opisuje zaprojektowaną architekturę systemu: jakie komponenty zostały wybrane, jak zostały ze sobą połączone i jakimi przesłankami kierowano się przy podejmowaniu kluczowych decyzji projektowych. Opis wdrożenia i szczegółów implementacji poszczególnych elementów zawarty jest w kolejnych rozdziałach.

3.1 Cele i ograniczenia projektowe

Projektując system, kierowano się czterema głównymi celami:

1. **Pełna automatyzacja** — dodanie nowych danych treningowych powinno wymagać wyłącznie wykonania polecenia `git push`; wszystkie dalsze kroki (walidacja, trening, ocena jakości, wdrożenie) powinny odbywać się bez udziału użytkownika.
2. **Reprodukowalność** — każda wersja modelu musi być jednoznacznie powiązana z wersją kodu, wersją danych treningowych i zapisanymi metrykami.
3. **Kontrola kosztów** — projekt realizowany jest w ramach akademickiego konta AWS z budżetem około 50 USD; zasoby obliczeniowe uruchamiane są tylko wtedy, gdy są faktycznie potrzebne.
4. **Kontrola czasu** — Laboratorium AWS może być uruchomione maksymalnie 4 godziny ciągiem, a trening modelu nie powinien przekraczać 2,5 godziny, aby umożliwić wykonanie pełnego cyklu CI/CD w ramach jednego uruchomienia.
5. **Porównywalność** — system musi umożliwiać zebranie mierzalnych danych (czas, koszt, liczba kroków manualnych) do porównania podejścia automatycznego z manualnym.

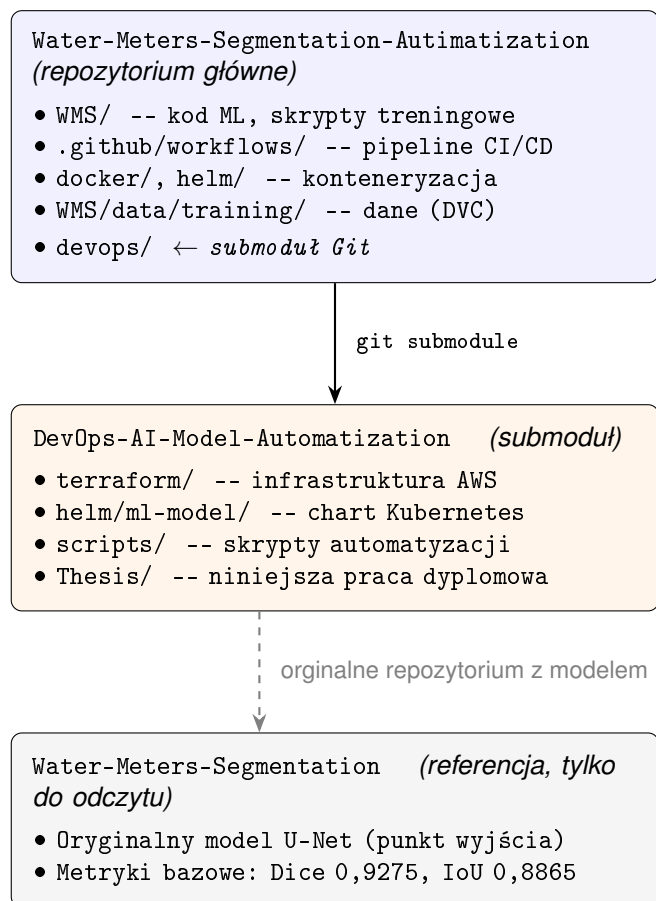
Ograniczenia budżetowe miały bezpośredni wpływ na wybór technologii. Wykluczone zostały usługi zarządzane o wysokim koszcie stałym: Amazon EKS (zarządzany Kubernetes \$150/mies.), Amazon RDS (zarządzana baza danych), NAT Gateway (\$32/mies.) oraz Application Load Balancer (\$18/mies.). W ich miejsce zastosowano tańsze odpowiedniki: k3s na pojedynczej instancji EC2 zamiast EKS, SQLite jako backend MLflow zamiast RDS, oraz dostęp przez publiczny adres IP z NodePort zamiast load balancera. Szacowany koszt systemu przy typowym obciążeniu deweloperskim (kilka uruchomień tygodniowo) wynosi około 11 USD miesięcznie zakładając lekki ruch.

3.2 Struktura repozytoriów

Projekt korzysta z dwóch repozytoriów Git połączonych mechanizmem submodułów:

- **Water-Meters-Segmentation-Autimatization** — repozytorium główne, zawiera kod ML, metadane danych treningowych śledzone przez DVC, workflow GitHub Actions, konfigurację Docker i Helm oraz skrypty pomocnicze. Całość procesu CI/CD wyzwalana jest w tym repozytorium.
- **DevOps-AI-Model-Automatization** — repozytorium infrastruktury, dołączone jako submoduł pod ścieżką `devops/`; zawiera moduły Terraform, skrypty konfiguracyjne EC2 oraz niniejszą pracę dyplomową.

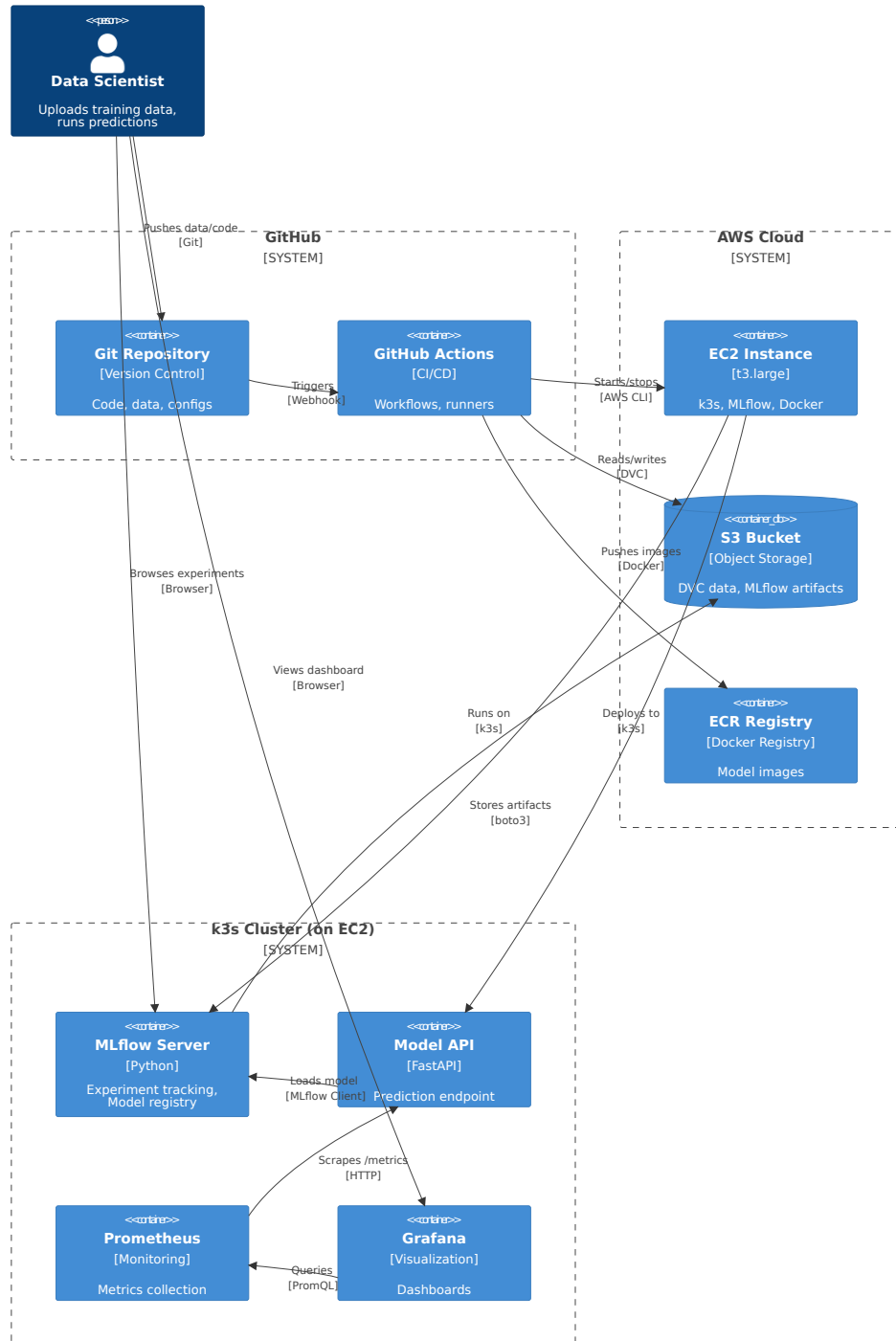
Podział na dwa repozytoria wynika z różnego rytmu zmian: kod ML i dane zmieniają się często i wyzwalają automatyczne procesy, natomiast definicja infrastruktury zmienia się rzadko i jest zarządzana ręcznie. Dzięki mechanizmowi submodułów konkretna wersja infrastruktury jest powiązana z konkretną wersją kodu aplikacji, co zachowuje pełną reprodukowalność środowiska.



Rys. 3 Struktura repozytoriów projektu: repozytorium główne zawiera infrastrukturę jako submoduł Git; repozytorium referencyjne dostarcza oryginalny model i metryki bazowe

3.3 Warstwy architektury

System zorganizowany jest w cztery logiczne warstwy, które razem tworzą kompletny pipeline MLOps. Rysunek 4 przedstawia ich wzajemne powiązania.



Rys. 4 Architektura systemu — cztery warstwy i ich komponenty

3.3.1 Warstwa kontroli wersji

Git i GitHub są centralnym punktem całego systemu — każda zmiana, czy to w kodzie, danych czy infrastrukturze, przechodzi przez repozytorium. Repozytorium przechowuje kod ML, definicje pipeline'u i infrastruktury, natomiast same pliki binarne danych treningowych przechowywane są w S3 i śledzone przez DVC. Mechanizm `pre-push hook` automatycznie kieruje nowe dane treningowe na oddzielną gałąź, inicjując cały dalszy proces. Szczegóły działania hooka opisano w rozdziale 5.

3.3.2 Warstwa automatyzacji

GitHub Actions orkiestruje cały cykl życia modelu. Reaguje na pojawienie się nowej gałęzi danych i sekwencyjnie wywołuje: walidację danych, start infrastruktury, trening, bramkę jakości, wdrożenie (jeśli model poprawił się) oraz sprzątanie. Workflow korzysta z reużywalnych składników dla operacji na EC2, co eliminuje powielanie konfiguracji. Szczegółowy opis pipeline'u zawarty jest w rozdziale 5.

3.3.3 Warstwa infrastruktury chmurowej

Cała infrastruktura AWS zdefiniowana jest jako kod w Terraform i tworzona powtarzalnie jednym poleceniem. Kluczowe zasoby to: instancja EC2 ze statycznym Elastic IP (niezbędnym przy cyklicznym stop/start), dwa zasobniki S3 (dane DVC i artefakty MLflow) oraz rejestr obrazów ECR. Instancja EC2 pełni podwójną rolę — runner treningu podczas pipeline'u i węzeł klastra k3s podczas serwowania modelu. Konfigurację infrastruktury opisuje rozdział 6.

3.3.4 Warstwa aplikacji i monitoringu

Na instancji EC2 działa klaster k3s z wdrożonym API modelu (FastAPI, zarządzany przez Helm), serwerem MLflow (systemd) oraz opcjonalnym stosem monitoringu Prometheus + Grafana. Model pobierany jest z rejestru MLflow przy starcie kontenera, co oddziela obraz Docker od konkretnej wersji modelu. Szczegóły konfiguracji wdrożenia opisuje rozdział 6.

3.4 Kluczowe decyzje projektowe

Tabela 1 zestawia najważniejsze decyzje projektowe podjęte podczas budowy systemu, wraz z uzasadnieniem wyboru i odrzuconą alternatywą. Każda z nich wynika z jednego lub więcej celów wymienionych w sekcji 3.

Tabela 1 Kluczowe decyzje architektoniczne

Decyzja	Wybór	Uzasadnienie
Orchestracja kontenerów	k3s na EC2	EKS jest 15× droższy; k3s wystarcza dla jednego węzła
Backend MLflow	SQLite	RDS \$30/mies.; SQLite jest wystarczający dla małego projektu; dane Meta przechowywane na EC2
Dostęp do API	NodePort 30080	Load Balancer \$18/mies.; publiczny adres IP EC2 z Elastic IP jest wystarczający
Tytuł instancji EC2	t3.large (8 GB RAM)	t3.small/medium: crash OOM przy pobieraniu obrazu PyTorch 8,69 GB; t3.large jest minimum
Stop/start EC2	Automatyczny po każdym pipeline	EC2 działa tylko 3–4 h na uruchomienie; oszczędność 80% kosztów vs. ciągłe działanie
Dane treningowe w Git	Tylko metadane DVC	Duże pliki binarne spowalniają Git i przekraczają limity; S3 jest tańszy i skalowalny
Trening przed PR	Trening → PR (jeśli lepszy)	PR tworzony przez bota nie wyzwala innych workflow (ograniczenie GitHub); kolejność eliminuje problem
Baseline bramki jakości	Dynamiczny z MLflow	Hardkodowany baseline staje się nieaktualny po każdym treningu; MLflow zawsze zna aktualny Production
IAM dla GitHub Actions	Tymczasowe sesji klucze	AWS Academy zabrania tworzenia ról IAM; OIDC niedostępne; klucze rotowane ręcznie

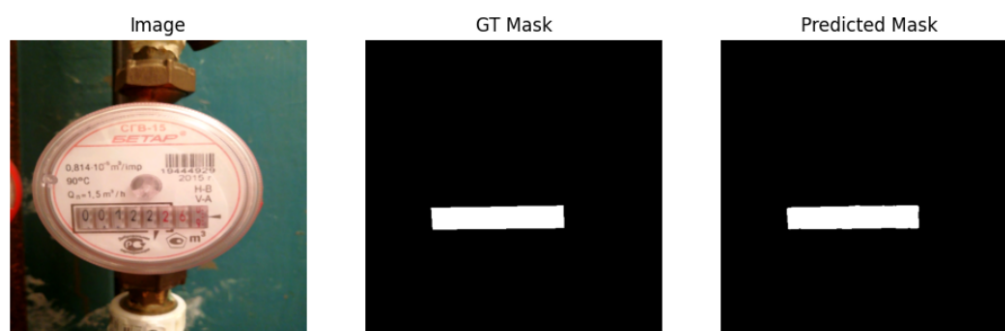
4 Model Sztucznej Inteligencji

4.1 Charakterystyka problemu segmentacji semantycznej

Model sztucznej inteligencji stanowiący podstawę niniejszego projektu został zaprojektowany do rozwiązywania zadania **segmentacji semantycznej obrazów wodomierzy**. Celem segmentacji jest przypisanie każdemu pikselowi obrazu etykiety określającej jego przynależność do jednej z wyróżnionych klas, w szczególności obszaru licznika, tarczy oraz tła.

Problem ten ma istotne znaczenie praktyczne w kontekście automatyzacji odczytu wskazań wodomierzy, gdzie poprawne wydzielenie obszaru licznika stanowi warunek konieczny dla dalszych etapów przetwarzania obrazu, takich jak rozpoznawanie cyfr lub analiza wskazań mechanicznych. W przeciwieństwie do klasycznej klasyfikacji obrazów, segmentacja semantyczna wymaga zachowania informacji przestrzennej oraz precyzyjnego odwzorowania granic obiektów, co znacząco zwiększa złożoność problemu.

Dodatkowym wyzwaniem są zmienne warunki akwizycji danych, obejmujące różnice w oświetleniu, kącie wykonania zdjęcia, jakości obrazu oraz stopniu zabrudzenia lub zużycia urządzeń pomiarowych. Czynniki te powodują, że klasyczne algorytmy przetwarzania obrazu okazują się niewystarczające, co uzasadnia zastosowanie metod opartych na uczeniu głębokim.



Rys. 5 Przykład predykcji modelu segmentacji semantycznej

4.2 Architektura modelu

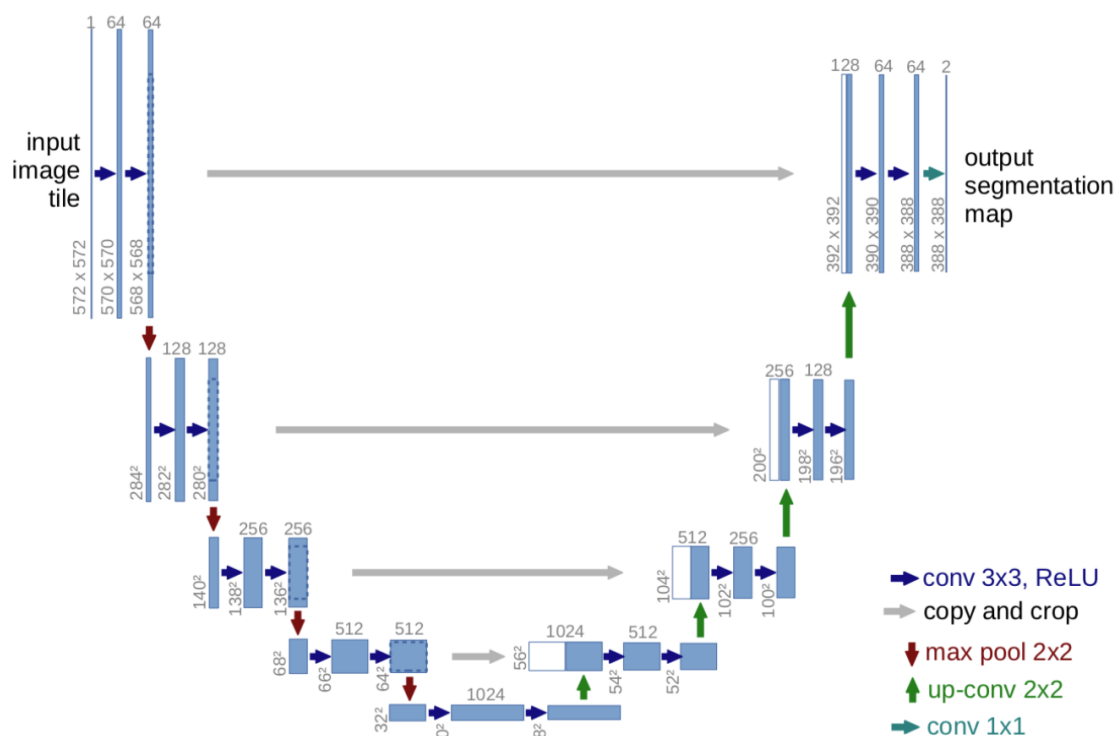
W projekcie wykorzystano konwolucyjną sieć neuronową opartą na architekturze **U-Net**, powszechnie stosowaną w zadaniach segmentacji semantycznej obrazów. Architektura ta charakteryzuje się symetryczną budową typu encoder-decoder, umożliwiającą jednocześnie wydobycie cech semantycznych oraz zachowanie informacji przestrzennej.

Część kodująca (encoder) odpowiada za stopniowe przekształcanie obrazu wejściowego do reprezentacji o coraz wyższym poziomie abstrakcji poprzez sekwencję operacji konwencji oraz redukcji rozdzielczości. Część dekodująca (decoder) realizuje proces rekonstrukcji obrazu wyjściowego, przywracając jego pierwotną rozdzielczość i generując maskę segmentacyjną.

Kluczowym elementem architektury są tzw. *skip connections*, umożliwiające bezpośrednie przekazywanie map cech pomiędzy odpowiadającymi sobie poziomami enkodera i dekodera.

Mechanizm ten pozwala ograniczyć utratę informacji lokalnej oraz poprawić jakość segmentacji, szczególnie w obszarach granicznych pomiędzy klasami.

Zastosowana implementacja architektury U-Net została dostosowana do specyfiki problemu, w tym rozdzielczości obrazów wejściowych oraz liczby klas segmentacyjnych. Model generuje maskę wyjściową o tej samej rozdzielczości co obraz wejściowy, co ułatwia dalsze etapy przetwarzania oraz ocenę jakości predykcji.



Rys. 6 Schemat architektury U-Net [4]

4.3 Dane treningowe i przygotowanie zbioru danych

Model został wytrenowany na zbiorze danych składającym się z obrazów wodomierzy oraz odpowiadających im masek. Każda próbka zawiera obraz wejściowy w postaci fotografii oraz maskę określającą przynależność poszczególnych pikseli do zdefiniowanych klas.

Proces przygotowania danych obejmował normalizację wartości pikseli, standaryzację rozdzielczości obrazów oraz podział zbioru danych na część treningową, walidacyjną i testową. W celu zwiększenia odporności modelu na zmienność danych wejściowych zastosowano techniki augmentacji danych, takie jak losowe obroty, skalowanie, przesunięcia oraz modyfikacje jasności i kontrastu.

Zastosowanie augmentacji pozwala ograniczyć ryzyko przeuczenia modelu oraz poprawić jego zdolność do generalizacji na dane niewidziane w procesie uczenia. Istotnym aspektem projektu jest fakt, że zbiór danych oraz jego kolejne wersje podlegają wersjonowaniu, co umożliwia jednoznaczne powiązanie konkretnej wersji modelu z użytymi danymi treningowymi.

Zbiór danych wykorzystany w projekcie pochodzi z platformy Kaggle [10].

4.4 Proces uczenia i metryki jakości

Proces uczenia modelu realizowany jest w sposób nadzorowany, z wykorzystaniem funkcji straty dostosowanej do problemu segmentacji semantycznej. Jako główne metryki oceny jakości modelu zastosowano **współczynnik Dice** oraz **Intersection over Union (IoU)**, które są standardowymi miarami stosowanymi w zadaniach segmentacyjnych.

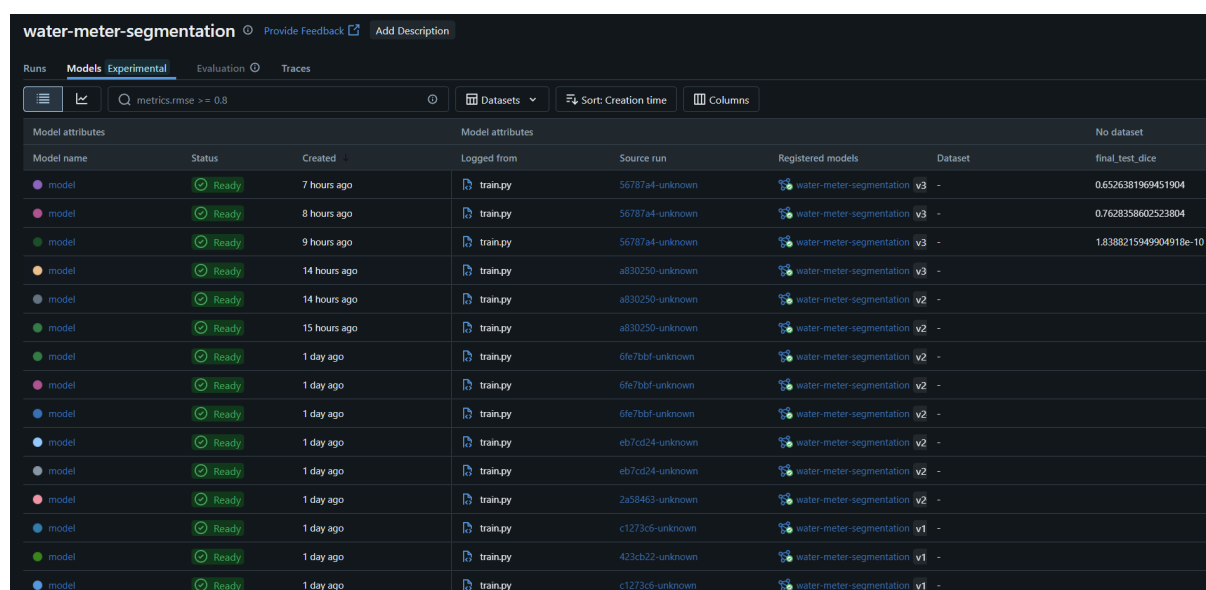
Współczynnik Dice umożliwia ocenę stopnia pokrycia obszaru przewidywanego przez model z obszarem referencyjnym, natomiast IoU mierzy stosunek części wspólnej obu obszarów do ich sumy. Zastosowanie obu metryk pozwala na bardziej kompleksową ocenę jakości predykcji, szczególnie w przypadku niebalansowanych klas.

Proces uczenia realizowany jest iteracyjnie, z wykorzystaniem zbioru walidacyjnego do monitorowania jakości modelu oraz ograniczania zjawiska przeuczenia. Najlepsza wersja modelu, zgodnie z przyjętymi kryteriami jakości, zapisywana jest jako artefakt i przekazywana do dalszych etapów zautomatyzowanego pipeline'u CI/CD.

4.5 Model jako artefakt w procesie CI/CD

W kontekście niniejszej pracy model sztucznej inteligencji traktowany jest jako pełnoprawny artefakt inżynierski, podlegający wersjonowaniu, testowaniu oraz kontrolowanemu wdrażaniu. Każda wersja modelu jest jednoznacznie powiązana z wersją kodu źródłowego, zestawem danych treningowych, konfiguracją hiperparametrów oraz uzyskanymi metrykami jakości.

Takie podejście umożliwia pełną reprodukowalność wyników oraz stanowi podstawę do dalszej analizy porównawczej pomiędzy podejściem manualnym a zautomatyzowanym procesem CI/CD. Szczegółowa analiza wpływu automatyzacji na jakość, stabilność oraz koszty utrzymania modeli sztucznej inteligencji zostanie przedstawiona w kolejnych rozdziałach pracy.



water-meter-segmentation							
Runs Models Experimental Evaluation Traces							
metrics.rmse >= 0.8							
Datasets Sort: Creation time Columns							
Model attributes				Model attributes			
Model name	Status	Created	Logged from	Source run	Registered models	Dataset	final_test_dice
model	Ready	7 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	0.6526381969451904
model	Ready	8 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	0.7628358602523804
model	Ready	9 hours ago	train.py	56787a4-unknown	water-meter-segmentation v3	-	1.8388215949904918e-10
model	Ready	14 hours ago	train.py	a830250-unknown	water-meter-segmentation v3	-	
model	Ready	14 hours ago	train.py	a830250-unknown	water-meter-segmentation v2	-	
model	Ready	15 hours ago	train.py	a830250-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	6fe7bbf-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	eb7cd24-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	eb7cd24-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	2a58463-unknown	water-meter-segmentation v2	-	
model	Ready	1 day ago	train.py	c1273c6-unknown	water-meter-segmentation v1	-	
model	Ready	1 day ago	train.py	423cb22-unknown	water-meter-segmentation v1	-	
model	Ready	1 day ago	train.py	c1273c6-unknown	water-meter-segmentation v1	-	

Rys. 7 Wersjonowanie modeli w rejestrze MLflow

5 Pipeline CI/CD

Rozdział opisuje implementację zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Prezentowane rozwiązanie realizuje przepływ: dodanie danych treningowych przez użytkownika wyzwala sekwencję kroków — walidację, trening, ocenę jakości i wdrożenie — bez konieczności ręcznej interwencji.

5.1 Hook pre-push

Pierwszym elementem pipeline jest hook Git pre-push [3], instalowany lokalnie na maszynie użytkownika przez skrypt `devops/scripts/install-git-hooks.sh`. Hook uruchamia się automatycznie przed każdym `git push` i sprawdza, czy w zestawie zmian znajdują się pliki w katalogach `WMS/data/training/images/` lub `WMS/data/training/masks/`.

Jeśli zmiany dotyczą danych treningowych, hook:

1. tworzy nową gałąź o nazwie `data/TIMESTAMP` (np. `data/20260215-143022`),
2. commituje do niej wyłącznie nowe pliki z katalogów danych,
3. przekierowuje push na tę gałąź zamiast na `main`.

Całość wykonuje się w mniej niż 10 sekund, ponieważ hook nie wymaga połączenia z AWS ani lokalnych poświadczeń chmurowych. Merge danych z historycznym zbiorem w S3 odbywa się po stronie GitHub Actions.

```
Training data changes detected. Creating data branch...
📦 Creating branch: data/20260217-013810
🚀 Pushing to data/20260217-013810...
🔍 Checking for training data changes...
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 16 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 584 bytes | 584.00 KiB/s, done.
Total 6 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'data/20260217-013810' on GitHub by visiting:
remote:   https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization/pull/new/data/20260217-013810
remote:
To https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git
 * [new branch]   data/20260217-013810 -> data/20260217-013810
branch 'data/20260217-013810' set up to track 'origin/data/20260217-013810'.

✅ Success! Your changes are now on branch: data/20260217-013810

Next steps:
 1. GitHub Actions will validate your data
 2. If valid, a Pull Request will be created automatically
 3. Training will run automatically
 4. If model improves, PR will be auto-approved
 5. You (or admin^ ) can then merge the PR

error: failed to push some refs to 'https://github.com/Rafallost/Water-Meters-Segmentation-Autimatization.git'
PS D:\school\bsc\Repositories\Water-Meters-Segmentation-Autimatization> |
```

Rys. 8 Wynik wypchania nowych danych na gałąź `data/TIMESTAMP`.

5.2 Walidacja danych

Po wypchnięciu gałęzi danych uruchamia się pierwszy job workflow `training-data-pipeline.yaml` [6]: `merge-and-validate`. Składa się on z dwóch faz.

Faza scalania

Job pobiera istniejące dane z Amazon S3 (za pomocą poświadczeń przechowywanych w sekretach GitHub), scala je z nowo dostarczonymi plikami i aktualizuje manifesty DVC. Scalanie jest idempotentne — pliki o identycznej nazwie i sumie kontrolnej nie są duplikowane.

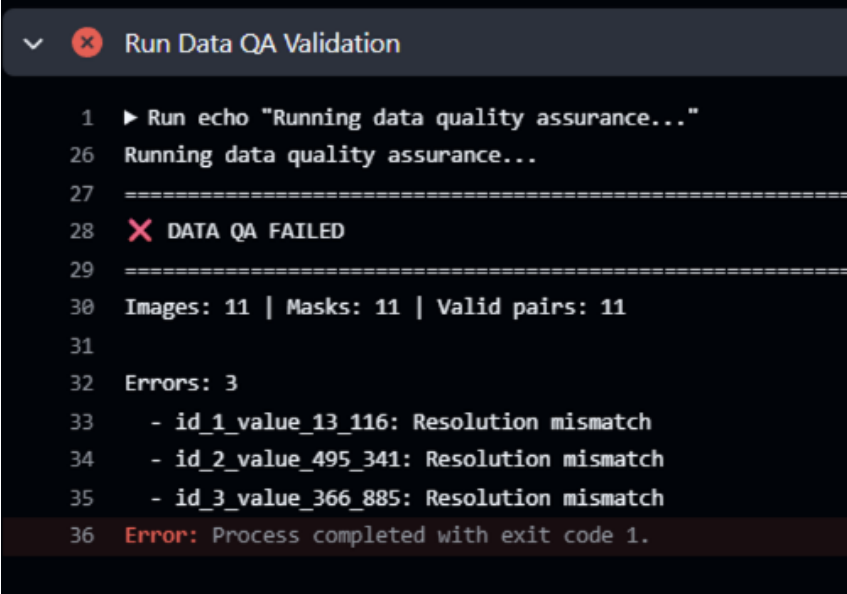
Faza walidacji

Skrypt `validate_data.py` weryfikuje scalony zbiór danych pod kątem czterech kryteriów:

- **kompletność par** — dla każdego obrazu w katalogu `images/` musi istnieć maska o tej samej nazwie w `masks/` (porównanie po *stem* nazwy pliku),
- **rozdzielczość** — każdy plik obrazu i maski musi mieć rozdzielczość dokładnie 512×512 px,
- **binarność masek** — maski mogą zawierać wyłącznie wartości 0 i 255 (brak półprzezroczystości),
- **format pliku** — akceptowane są formaty `.jpg` oraz `.png`.

Wynik walidacji jest publikowany jako komentarz w pull requestcie. W przypadku błędów komentarz zawiera listę plików niezgodnych z wymaganiami, a dalsze joby są pomijane (warunkowanie przez `needs` i `if: needs.merge-and-validate.outputs.valid == 'true'`).

```
# Fragment skryptu walidacji
for img_stem in image_stems:
    if img_stem not in mask_stems:
        errors.append(f"Brak maski dla: {img_stem}")
```

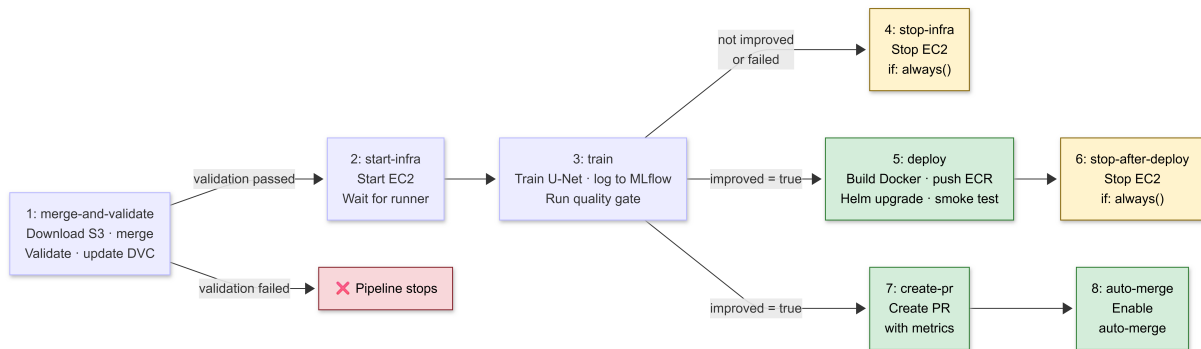


```
1 ▶ Run echo "Running data quality assurance..."
26 Running data quality assurance...
27 =====
28 X DATA QA FAILED
29 =====
30 Images: 11 | Masks: 11 | Valid pairs: 11
31
32 Errors: 3
33 - id_1_value_13_116: Resolution mismatch
34 - id_2_value_495_341: Resolution mismatch
35 - id_3_value_366_885: Resolution mismatch
36 Error: Process completed with exit code 1.
```

Rys. 9 Przykład komentarza z wynikiem walidacji danych. W tym przypadku wykryto złą rozdzielczość, co skutkuje zatrzymaniem dalszych kroków pipeline.

5.3 Pipeline treningowy

Główny workflow `training-data-pipeline.yaml` składa się z ośmiu jobów. Joby 1–3 wykonywane są sekwencyjnie. Po zakończeniu treningu workflow rozgałęzia się: gdy model się nie poprawił, uruchamia się `stop-infra` (job 4); gdy model się poprawił — równolegle uruchamiają się `deploy` (job 5) i `create-pr` (job 7). Job 6 zatrzymuje EC2 po wdrożeniu, a job 8 finalizuje scalenie.



Rys. 10 Sekwencja jobów w workflow `training-data-pipeline.yaml`

Job 1: merge-and-validate

Opracowane we wcześniejszym fragmencie. Pobiera istniejące dane z S3, scala je z nowo dostarczonymi plikami, uruchamia walidację, aktualizuje manifesty DVC i przesyła scalony zbiór jako artefakt GitHub Actions..

Job 2: start-infra

Wykonuje się na hoście GitHub (nie na EC2) i jest odpowiedzialny wyłącznie za uruchomienie infrastruktury. Wywołuje AWS CLI (`aws ec2 start-instances`) z identyfikatorem instancji zapisanym w sekretach repozytorium. Po wysłaniu polecenia co 30 sekund sprawdza API GitHub, czy self-hosted runner przypisany do tej instancji pojawił się w stanie *online* — czeka maksymalnie 10 minut. Dzięki temu każdy kolejny job pewnie trafi na już działającą instancję.

Job 3: train

Jedyny job wykonywany na self-hosted runnerze, czyli bezpośrednio na instancji EC2. Pobiera scalony zbiór danych z artefaktu GitHub Actions, uruchamia skrypt `train.py`, który inicjalizuje model U-Net z seedem równym numerowi uruchomienia (`run_number`), trenuje przez zdefiniowaną liczbę epok i loguje metryki `val_dice` oraz `val_iou` do MLflow. Po zakończeniu treningu skrypt `quality_gate.py` pobiera metryki aktualnego modelu Production z rejestru MLflow, porównuje je z wynikami nowego treningu i ustawia output `improved=true` lub `improved=false`, sterując dalszym przebiegiem workflow.

Job 4: stop-infra

Zatrzymuje instancję EC2 gdy trening zakończył się błędem lub model nie poprawił wyników (`if: always() && (train.result != 'success' || improved != 'true')`). Jest to reużywalny workflow wywołujący `aws ec2 stop-instances`. Dzięki warunkowi `always()` job uruchamia się nawet gdy poprzednie jobe zakończyły się wyjątkiem, eliminując ryzyko pozostawienia uruchomionej instancji i nieoczekiwanych kosztów.

Job 5: deploy

Uruchamia się wyłącznie jeśli model spełnił kryteria bramki jakości (`if: improved == 'true'`). Wykonuje się na self-hosted runnerze na EC2. Buduje obraz Docker na podstawie `docker/Dockerfile.serve`, taguje go numerem wersji i hashem commita, uwierzytelnia się w Amazon ECR i wgrywa obraz. Następnie wywołuje `helm upgrade`, który zaktualizuje deployment na klastrze k3s do nowej wersji obrazu — kontener przy starcie pobiera model ze stadium Production z rejestru MLflow, co oddziela wersję obrazu od wersji modelu. Po wdrożeniu wykonywany jest smoke test sprawdzający dostępność endpointów `/health`, `/predict` i `/metrics`.

Job 6: stop-after-deploy

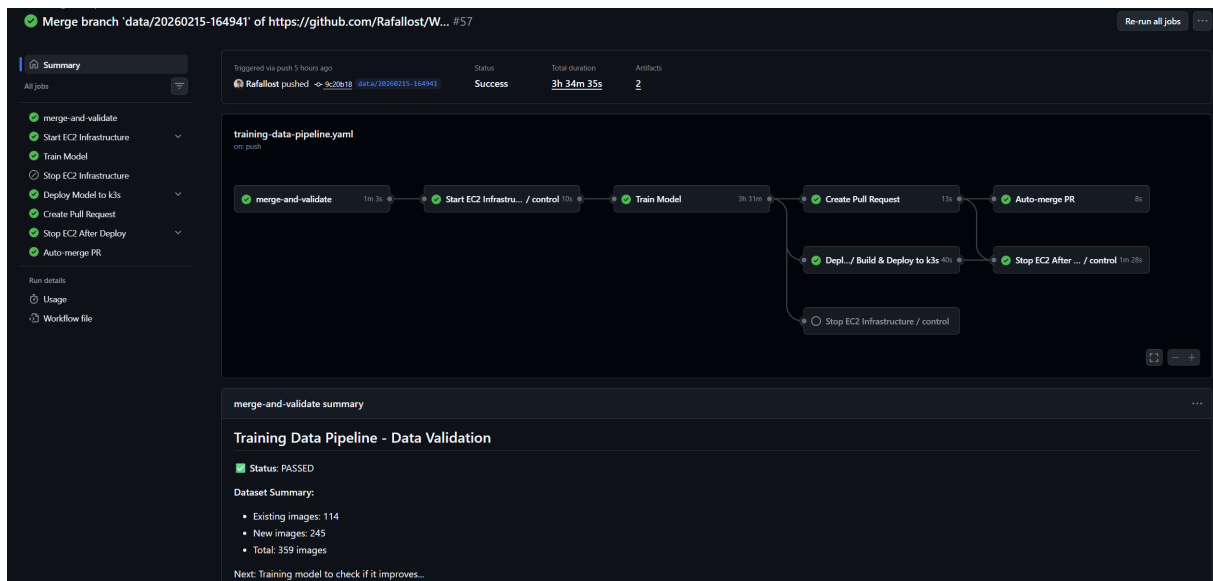
Zatrzymuje instancję EC2 po zakończeniu joba `deploy` (`if: always()`). Stanowi odpowiednik `stop-infra` dla ścieżki, gdy model się poprawił. Istnieje jako osobny job, ponieważ `deploy` korzysta z runnera na EC2 — instancja musi pozostać uruchomiona do końca wdrożenia i może zostać zatrzymana dopiero po jego zakończeniu.

Job 7: create-pr

Uruchamia się równolegle z jobem `deploy`, jeśli model spełnił kryteria bramki jakości. Tworzy pull request z gałęzi danych do `main` z automatycznie generowanym opisem zawierającym zestawienie metryk: wartości `val_dice` i `val_iou` nowego modelu oraz poprzedniego baseline, a także link do uruchomienia MLflow. Pull request scala zaktualizowane manifesty DVC (`images.dvc`, `masks.dvc`) z gałęzią główną, trwale łącząc wersję kodu z wersją danych.

Job 8: auto-merge

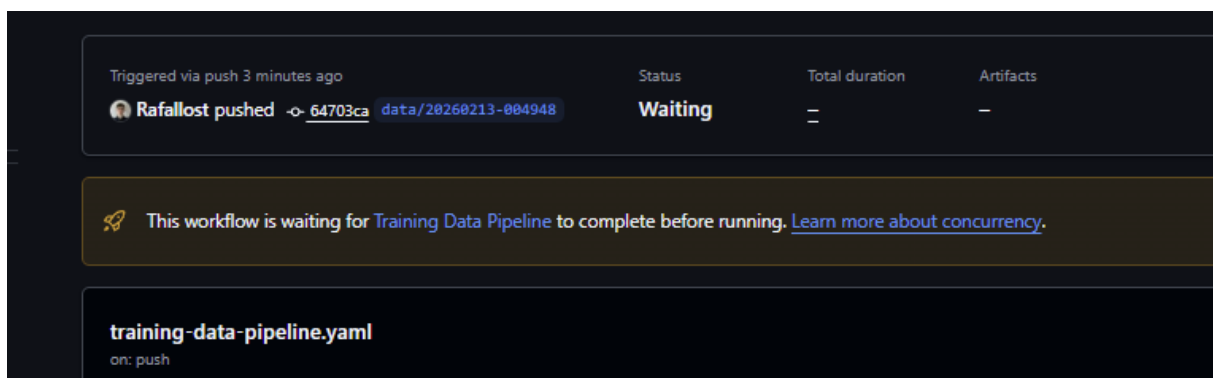
Włącza automatyczne scalanie pull requesta przez API GitHub (`gh pr merge -auto`). Scalenie następuje natychmiast po pozytywnym przejściu statusowych sprawdzeń wymaganych przez reguły ochrony gałęzi. W efekcie, jeśli model się poprawił, cały cykl — od wypchnięcia danych do scalenia z `main` — odbywa się bez żadnej ręcznej interwencji.



Rys. 11 Udany przebieg pipeline CI/CD z automatycznym wdrożeniem nowej wersji modelu.

5.4 Współbieżność i kolejkovanie

Workflow skonfigurowany jest z kluczem `concurrency.cancel-in-progress: false`. Oznacza to, że jeśli użytkownik wypchnął dane kilkakrotnie zanim poprzednie uruchomienie zdążyło się zakończyć, kolejne uruchomienia nie są anulowane — trafiają do kolejki i wykonują się po zakończeniu poprzednich. Podejście to chroni przed sytuacją, w której późniejszy push nadpisze wyniki wcześniejszego treningu, zanim zostanie on oceniony przez bramkę jakości.



Rys. 12 Kolejkovanie uruchomień workflow w GitHub Actions — nowe uruchomienie czeka na zakończenie poprzedniego.

5.5 Bramka jakości

Bramka jakości jest mechanizmem decydującym, czy nowy model jest wystarczająco dobry, żeby trafić do produkcji. Implementacja jest w skrypcie `quality_gate.py` i korzysta z API MLflow Model Registry [16].

Pobieranie baseline

Skrypt łączy się z MLflow przez `MlflowClient` i pobiera metryki aktualnego modelu Production:

```

client = MlflowClient(tracking_uri=mlflow_uri)
versions = client.get_latest_versions(
    "water-meter-segmentation",
    stages=["Production"]
)
if versions:
    baseline_run = client.get_run(versions[0].run_id)
    baseline_dice = float(
        baseline_run.data.metrics["val_dice"]
    )
else:
    baseline_dice = 0.0 # pierwsze trenowanie

```

Logika decyzyjna

Model jest uznany za ulepszony, jeśli obie metryki walidacyjne są wyższe od baseline:

- `val_dice > baseline_dice`
- `val_iou > baseline_iou`

Jeśli warunek jest spełniony, skrypt promuje nowy model do etapu Production w MLflow i archiwizuje poprzednią wersję. Wynik jest przekazywany jako output do GitHub Actions (`echo "improved=true" >> $GITHUB_OUTPUT`).

5.6 Automatyczne wdrożenie

Gdy bramka jakości potwierdzi poprawę modelu, job `deploy` w tym samym workflow buduje i wdraża nowy obraz Docker na klastrze k3s — równolegle z tworzeniem pull requesta (job `create-pr`).

Budowanie obrazu

Obraz jest budowany na podstawie pliku `docker/Dockerfile.serve`. Używa warstwy bazowej z PyTorch CPU, kopiuje kod API i instaluje zależności z `requirements.txt`. Po zbudowaniu obraz jest tagowany numerem wersji i hashem commita, a następnie wysyłany do Amazon ECR.

Wdrożenie Helm

Helm [7] aktualizuje deployment na klastrze k3s [18] nową wersją obrazu. Konfiguracja wdrożenia zdefiniowana jest w `infrastructure/helm-values.yaml`. Po wdrożeniu automatycznie wykonywany jest smoke test sprawdzający dostępność endpointów `/health`, `/predict` i `/metrics`.

6 Wdrożenie Systemu

Rozdział opisuje infrastrukturę, na której działa system, sposób jej konfiguracji oraz scenariusze użycia z perspektywy użytkownika.

6.1 Infrastruktura Terraform

Infrastruktura AWS jest zdefiniowana jako kod przy użyciu Terraform [1]. Definicje znajdują się w katalogu `devops/terraform/`. Zastosowana architektura minimalizuje koszty — używa wyłącznie zasobów niezbędnych do działania projektu i unika kosztownych komponentów zarządzanych przez AWS (EKS, RDS, NAT Gateway, Load Balancer).

Tworzone zasoby:

- **VPC i podsieć publiczna** — prosta sieć z bezpośrednim dostępem do internetu przez Internet Gateway; eliminuje potrzebę NAT Gateway,
- **Security Group** — otwiera porty SSH (22), HTTP (80), NodePort Kubernetes (30000–32767) i port MLflow (5000) dla dozwolonych adresów IP,
- **Instancja EC2** — uruchamiana na żądanie przez workflow [6]; typ instancji konfigurowalny przez zmienną (`t3.xlarge` dla produkcji ML),
- **Amazon S3** — dwa bucket'y: jeden dla artefaktów MLflow, drugi opcjonalnie dla danych treningowych DVC,
- **Amazon ECR** — rejestr obrazów Docker dla API modelu,
- **IAM Role** — rola instancji EC2 z uprawnieniami do odczytu i zapisu S3 oraz ECR.

Infrastruktura uruchamiana jest jednorazowo poleceniem `terraform apply`. Instancja EC2 jest następnie zatrzymywana i uruchamiana automatycznie przez workflow tylko na czas treningu i wdrożenia, co istotnie obniża koszty eksploatacji.

6.2 Konfiguracja EC2 i k3s

Konfiguracja instancji EC2 odbywa się automatycznie przy pierwszym uruchomieniu za pomocą skryptu `user-data`. Skrypt jest szablonowany przez Terraform i zawiera zmienne takie jak adres bucket'a S3 czy region.

Kroki wykonywane przez `user-data`:

1. Instalacja zależności systemowych (`dnf install -y git curl libicu`).
2. Instalacja k3s — uproszczonego Kubernetes [18].
3. Instalacja MLflow i boto3 przez pip: `pip3 install -ignore-installed mlflow boto3`.
4. Konfiguracja MLflow jako usługi systemd:

```
[Service]
ExecStart=mlflow server \
  --host 0.0.0.0 --port 5000 \
  --default-artifact-root s3://${mlflow_bucket}/
```

5. Instalacja GitHub Actions runnera jako usługi systemd.

Instancja korzysta z Amazon Linux 2023, który oferuje nowszy OpenSSL 3.0 i lepszy ekosystem pakietów niż poprzednia wersja Amazon Linux 2. Po zakończeniu `user-data` instancja jest gotowa do przyjęcia zadań z GitHub Actions bez dalszej konfiguracji ręcznej.

6.3 Konfiguracja Helm

API modelu wdrażane jest na klaster k3s za pomocą Helm [7]. Chart konfiguruje następujące zasoby Kubernetes:

- **Deployment** — jedna replika kontenera z API; konfiguruje żądania i limity CPU/RAM,
- **Service (NodePort)** — udostępnia API na porcie 30080 węzła EC2; eliminuje potrzebę Load Balancera,
- **ConfigMap** — zmienne środowiskowe aplikacji (URI MLflow, ścieżki do modelu),
- **Secret** — poświadczenia AWS do pobrania modelu z S3.

Kluczowe wartości w pliku `infrastructure/helm-values.yaml`:

```
image:
  repository: <account>.dkr.ecr.us-east-1.amazonaws.com/model-api
  tag: latest
env:
  MLFLOW_TRACKING_URI: http://10.0.1.x:5000
  MODEL_STAGE: Production
resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
```

Po wdrożeniu API jest dostępne pod adresem `http://<ec2-public-ip>:30080`. Dostępne endpointy to `/health` (status usługi), `/predict` (predykcja segmentacji) i `/metrics` (metryki Prometheus).

6.4 Stos monitorowania

Monitoring oparty na Prometheus [17] i Grafana wdrażany jest jako oddzielny stos na tym samym klastrze k3s. Składa się z trzech komponentów:

- **Prometheus** — zbiera metryki z endpointu `/metrics` API modelu co 15 sekund; konfiguracja scraping w `prometheus.yml`,
- **Grafana** — wizualizuje metryki na predefiniowanych dashboardach; dostępna na porcie NodePort 30300,
- **kube-state-metrics** — dostarcza metryki Kubernetes (stan podów, zużycie zasobów węzła).

API modelu eksponuje metryki w formacie Prometheus przy użyciu biblioteki `prometheus_client`. Śledzone metryki to:

- `prediction_requests_total` — łączna liczba żądań predykcji,

- `prediction_duration_seconds` — histogram czasów odpowiedzi,
- `prediction_errors_total` — liczba błędów.

6.5 Workflow użytkownika

6.5.1 Scenariusz 1: Dodanie nowych danych treningowych

Jest to główny scenariusz użycia systemu. Użytkownik chcący dostarczyć nowe obrazy wodomierzy wykonuje następujące kroki:

1. Umieszcza pliki obrazów w `WMS/data/training/images/` i odpowiadające im maski w `WMS/data/training/masks/`.
2. Dodaje pliki do staging area Git: `git add WMS/data/training/`.
3. Tworzy commit: `git commit -m "data: add N new training samples"`.
4. Wywołuje `git push`.

Hook pre-push automatycznie przekierowuje push na gałąź `data/TIMESTAMP`. Dalej system działa bez udziału użytkownika: walidacja, trening, bramka jakości, ewentualne scalenie. Użytkownik może śledzić postęp w zakładce Actions repozytorium GitHub.

6.5.2 Scenariusz 2: Predykcja lokalna

Użytkownik może uruchomić predykcję na nowych obrazach bez uruchamiania infrastruktury chmurowej:

1. Przy pierwszym użyciu pobiera aktualny model Production:

```
python WMS/scripts/sync_model_aws.py
```

2. Umieszcza obrazy do segmentacji w `WMS/data/predict/input/`.
3. Uruchamia predykcję: `python WMS/predicts.py`
4. Wyniki zapisywane są w `WMS/data/predict/output/`.

Skrypt `sync_model_aws.py` pobiera model zarejestrowany w MLflow jako Production i zapisuje go lokalnie. Model jest przechowywany w katalogu ignorowanym przez Git, więc nie trafia do repozytorium.

6.5.3 Scenariusz 3: Ręczne uruchomienie treningu

W sytuacjach wyjątkowych (np. zmiana hiperparametrów bez nowych danych) trening można uruchomić ręcznie przez zakładkę *Actions* w GitHub, wybierając workflow `train.yml` i klikając *Run workflow*. Ten tryb wymaga podania parametrów: branch'u z danymi oraz opcjonalnie liczby epok. Wynik treningu jest rejestrowany w MLflow, ale nie tworzy automatycznego pull requesta — decyzja o wdrożeniu należy do użytkownika.

7 Analiza Wyników i Działania Systemu WIP

7.1 Metodologia porównania

Celem eksperymentów jest ilościowe i jakościowe porównanie trzech podejść do zarządzania cyklem życia modelu uczenia maszynowego [11]:

1. **Podejście ręczne** — wszystkie kroki (przygotowanie danych, trening, walidacja, wdrożenie) wykonywane przez człowieka bez automatyzacji.
2. **Uproszczone CI/CD** — pipeline z częściową automatyzacją (automatyczny trening po merge, ręczna walidacja jakości i wdrożenie).
3. **Zautomatyzowane CI/CD (niniejszy projekt)** — pełna automatyzacja od push danych do wdrożenia modelu.

Porównanie obejmuje następujące wymiary:

- czas potrzebny do wykonania każdego scenariusza,
- jakość modelu (metryki Dice i IoU),
- pracochłonność (liczba ręcznych kroków),
- ryzyko błędu ludzkiego,
- skalowalność procesu.

7.2 Czasy wykonania scenariuszy

Tabela 2 przedstawia zmierzone czasy dla każdego podejścia w scenariuszu dodania nowej partii danych treningowych i wdrożenia ulepszanego modelu.

Tabela 2 Porównanie czasów wykonania scenariuszy

Etap	Ręczny	Uproszczone CI/CD	Zautomatyzowane CI/CD
Przygotowanie danych	[uzup.]	[uzup.]	< 1 min
Walidacja danych	[uzup.]	[uzup.]	[uzup.]
Trening modelu	[uzup.]	[uzup.]	[uzup.]
Ocena jakości	[uzup.]	[uzup.]	< 1 min
Wdrożenie	[uzup.]	[uzup.]	[uzup.]
Łącznie	[uzup.]	[uzup.]	[uzup.]

7.3 Metryki modelu

Tabela 3 przedstawia metryki modelu dla kolejnych wersji wytrenowanych w ramach eksperymentów. Baseline pochodzi z repozytorium referencyjnego *Water-Meters-Segmentation* [10]. Metryki są rejestrowane automatycznie przez MLflow [19] podczas każdego treningu.

Tabela 3 Metryki jakości modelu dla kolejnych wersji

Wersja modelu	Dice	IoU	Uwagi
Baseline (referencyjny)	0,9275	0,8865	Oryginalny model
v1 — pierwsze trenowanie	[uzup.]	[uzup.]	[uzup.]
v2 — po dodaniu danych	[uzup.]	[uzup.]	[uzup.]

7.4 Metryki infrastruktury

7.4.1 Zużycie zasobów podczas treningu

Podczas treningu modelu monitorowano obciążenie instancji EC2. Kluczowe obserwacje:

- szczytowe zużycie CPU:
- szczytowe zużycie RAM:
- czas treningu jednej epoki:

7.4.2 Zużycie zasobów podczas serwowania

API modelu serwującego predykcje charakteryzuje się następującymi parametrami wydajnościowymi:

- średni czas odpowiedzi dla jednego obrazu:
- przepustowość:
- zużycie RAM przez pod:

7.4.3 Koszty infrastruktury AWS

Tabela 4 Szacowane koszty infrastruktury AWS

Zasób	Koszt	Uwagi
EC2 t3.large (trening)	[uzup.] USD/h	Uruchamiana tylko podczas treningu
EC2 t3.large (serwowanie)	[uzup.] USD/h	Stop/start przez workflow
S3 (artefakty)	[uzup.] USD/mies.	Zależy od liczby modeli
ECR (obrazy)	[uzup.] USD/mies.	
Łącznie (projekt)	[uzup.] USD	Za cały okres realizacji

7.5 Ocena jakościowa

7.5.1 Łatwość wdrożenia

7.5.2 Stabilność systemu

W trakcie testów zaobserwowano następujące zachowania systemu w sytuacjach wyjątkowych:

- **Niepoprawne dane** — walidator odrzucał dane z komunikatem i nie uruchamiał treningu,
- **Model gorszy od baseline** — bramka jakości odrzucała model bez tworzenia PR,
- **Awaria instancji EC2** — job stop-infra uruchamiał się zawsze (if: always()), zapobiegając wyciekom kosztów.

7.5.3 Skalowalność

Zastosowane podejście ma następujące ograniczenia skalowalności:

- single-node k3s — brak wysokiej dostępności,
- jeden runner EC2 — brak równoległych treningów,
- model proof of concept — nie testowano przy zbiorach > 1000 obrazów.

8 Podsumowanie i wnioski WIP

8.1 Realizacja celów pracy

Niniejsza praca stawiała sobie za cel zaprojektowanie, implementację i ocenę zautomatyzowanego pipeline CI/CD dla modeli uczenia maszynowego. Poniżej zestawiono cele sformułowane we wprowadzeniu z oceną stopnia ich realizacji.

- **Zaprojektowanie architektury systemu automatyzacji** —
- **Implementacja pipeline CI/CD integrującego walidację danych, trening i wdrożenie** —
- **Wprowadzenie obiektywnej bramki jakości opartej na metrykach ML** —
- **Porównanie podejścia ręcznego z zautomatyzowanym** —
- **Realizacja w ramach budżetu AWS Academy (~\$50)** —

8.2 Wnioski

Na podstawie przeprowadzonych eksperymentów i obserwacji sformułowano następujące wnioski. Kontekst porównawczy oparty jest na literaturze z zakresu DevOps [8] oraz MLOps [11].

1. Automatyzacja pipeline CI/CD dla modeli ML istotnie skraca czas potrzebny do wdrożenia nowej wersji modelu w porównaniu do podejścia ręcznego.
2. Wprowadzenie dynamicznej bramki jakości opartej na metrykach MLflow eliminuje ryzyko przypadkowego wdrożenia modelu gorszego od aktualnie produkcyjnego, co jest szczególnie istotne przy niedeterministycznym procesie trenowania.
3. Wersjonowanie danych za pomocą DVC [9] zapewnia pełną odtwarzalność eksperymentów — dla dowolnej wersji modelu możliwe jest odtworzenie dokładnego zbioru danych, na którym był trenowany.
4. Zastosowanie k3s zamiast zarządzanego Kubernetes (EKS) okazało się wystarczające dla projektu proof of concept i pozwoliło znacząco ograniczyć koszty infrastruktury.
5. Architektura stop/start instancji EC2 jest efektywna kosztowo, ale wprowadza opóźnienie na uruchomienie instancji i runnera (~3–5 minut). W środowisku produkcyjnym z wymaganiami SLA należałoby rozważyć utrzymywanie instancji aktywnej.

8.3 Ograniczenia

Projekt realizowany był jako proof of concept z następującymi świadomymi ograniczeniami:

- **Budżet AWS Academy** — sesje laboratoryjne wygasają co 4 godziny, IAM role mają ograniczone uprawnienia (m.in. brak `s3:PutObject` dla lokalnych poświadczeń). Wymusiło to przeniesienie wszystkich operacji S3 do GitHub Actions.
- **Single-node k3s** — brak wysokiej dostępności. Awaria instancji EC2 powoduje niedostępność całego systemu.
- **Mały zbiór danych** — eksperyment przeprowadzono na zbiorze
- **Jeden model** — architektura jest zaprojektowana pod konkretny model (U-Net dla segmentacji). Generalizacja na inne typy modeli wymagałaby modyfikacji skryptów walidacji

i bramki jakości.

8.4 Kierunki dalszego rozwoju

System stanowi solidną bazę, którą można rozbudować w kilku kierunkach:

- **Multi-node Kubernetes** — zastąpienie single-node k3s klastrem zapewniającym wysoką dostępność i możliwość skalowania poziomego.
- **Experiment tracking** — rozbudowa wykorzystania MLflow o śledzenie hiperparametrów i automatyczne porównywanie wielu eksperymentów.
- **Data drift monitoring** — dodanie mechanizmu wykrywającego rozbieżność między danymi treningowymi a produkcyjnymi.
- **A/B testing** — wdrożenie mechanizmu stopniowego rollout nowej wersji modelu z automatycznym rollback przy pogorszeniu metryk.
- **Generalizacja** — abstrakcja pipeline pod inne typy modeli i zadań ML, nie tylko segmentację obrazów.

Literatura

- [1] Yevgeniy Brikman. *Terraform: Up and Running*. O'Reilly Media, 3 edition, 2022. ISBN 978-1-098-11674-3. <https://cdn.bookey.app/files/pdf/book/en/terraform.pdf>.
- [2] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and kubernetes. In *ACM Queue*, volume 14, pages 70–93, 2016. doi: 10.1145/2898442.2898444. <https://dl.acm.org/doi/10.1145/2898442.2898444>.
- [3] Scott Chacon and Ben Straub. *Pro git*. <https://git-scm.com/book>, 2014. Dostęp: 10.02.2026.
- [4] DataFight. U-net i segmentacja obrazu. <https://datafight.wordpress.com/2020/05/04/u-net-i-segmentacja-obrazu/>, 2020. Dostęp: 10.02.2026.
- [5] MLOPS Flow Diagram. Mlops lifecycle diagram. <https://bluemetrika.com/czym-jest-mlops/>, 2024. Dostęp: 16.02.2026.
- [6] GitHub Inc. Github actions documentation. <https://docs.github.com/en/actions>, 2024. Dostęp: 10.02.2026.
- [7] Helm Authors. Helm — the kubernetes package manager. <https://helm.sh/docs/>, 2023. Dostęp: 10.02.2026.
- [8] Jez Humble and David Farley. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN 978-0-321-60191-9. <https://www.informit.com/store/continuous-delivery-9780321601919>.
- [9] Iterative.ai. Data version control — open-source version control system for machine learning projects. <https://dvc.org/doc>, 2023. Dostęp: 10.02.2026.
- [10] Kaggle. Yandex toloka water meters dataset. <https://www.kaggle.com/datasets/tapakah68/yandextoloka-water-meters-dataset>, 2023. Dostęp: 10.02.2026.
- [11] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023. doi: 10.1109/ACCESS.2023.3262138.
- [12] Grafana Labs. Grafana documentation: Introduction. <https://grafana.com/docs/grafana/latest/fundamentals/>, 2024. Dostęp: 16.02.2026.
- [13] Mike Loukides. What is devops? <https://cdn.bookey.app/files/pdf/book/en/what-is-devops-.pdf>, 2012. Bookey summary PDF, accessed: 2026-02-16.
- [14] Dirk Merkel. Docker: Lightweight linux containers for consistent development and deployment. *Linux Journal*, 2014(239), 2014. <https://www.linuxjournal.com/content/docker-lightweight-linux-containers-consistent-development-and-deployment>.

- [15] Nehal Navlani. Ci/cd pipeline - system design. <https://www.geeksforgeeks.org/system-design/cicd-pipeline-system-design/>, 2025.
- [16] MLflow Project. MLflow model registry. <https://mlflow.org/docs/latest/ml/model-registry/>, 2024. Dostęp: 16.02.2026.
- [17] Prometheus Authors. Prometheus — monitoring system and time series database. <https://prometheus.io/docs/introduction/overview/>, 2023. Dostęp: 10.02.2026.
- [18] Rancher Labs. k3s — lightweight kubernetes. <https://docs.k3s.io>, 2023. Dostęp: 10.02.2026.
- [19] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Aarth Singh, et al. MLflow: A system for machine-learning lifecycle management. In *Proceedings of the Workshop on Data Management for End-to-End Machine Learning (DEEM)*. ACM, 2018. doi: 10.1145/3399579.3399604.